LANGGRAPH PLAYWRIGHT CODE GENERATOR

1. Purpose

The purpose of this project is to demonstrate how LangGraph can be used to build an agentic workflow that generates, saves, and executes Playwright automation code using a Large Language Model (Google Gemini).

This script acts as an AI-powered Playwright code generator and runner, where natural language instructions are converted into executable browser automation tests.

2. Description

This project implements a multi-node LangGraph workflow that:

- Accepts natural language automation instructions
- Uses Gemini to generate a complete Playwright Python script
- Saves the generated script to a file
- Automatically executes the script using Python

The workflow is fully automated and demonstrates code generation + execution orchestration using LangGraph.

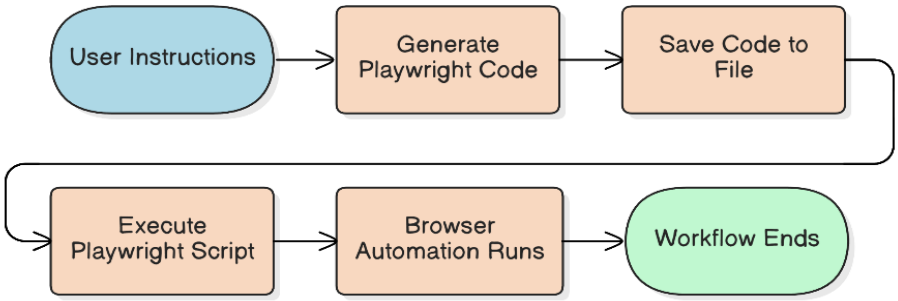
3. Workflow Overview

The workflow follows a sequential, multi-step execution model:

1. User provides Playwright test instructions
2. Gemini generates a full Playwright Python script
3. The generated code is saved to a file
4. The saved script is executed automatically
5. The workflow terminates

4. Architecture Overview

LangGraph Workflow



Agent State

The workflow uses a shared state object to pass data between nodes.

Field	Description
instructions	Natural language automation instructions
playwright_code	Generated Playwright Python code
file_path	Path of the saved Playwright script

5. Tech Stack

- **Language:** Python
- **Agent Framework:** LangGraph
- **LLM Framework:** LangChain
- **Model Provider:** Google Gemini
- **Model Used:** gemini-3-flash-preview
- **Automation Tool:** Playwright (Python)
- **Execution:** Python subprocess
- **Interface:** Script-based execution

6. Setup

Dependencies Installation

```
pip install langgraph langchain langchain-google-genai playwright
playwright install
```

API Key Configuration

Set the Google Gemini API key as an environment variable.

- **macOS / Linux**

```
export GOOGLE_API_KEY="your_api_key_here"
```

- **Windows (PowerShell)**

```
setx GOOGLE_API_KEY "your_api_key_here"
```

7. Key Components

7.1 Agent State Definition

Defines the shared state passed between LangGraph nodes.

7.2 Playwright Code Generator Node

- Sends system and user prompts to Gemini
- Forces output to be **valid Python code only**
- Cleans markdown formatting if present
- Stores generated code in the state

7.3 Save Code Node

- Writes the generated Playwright script to a file
- Stores the file path in the state
- Prints the generated code for visibility

7.4 Execute Code Node

- Executes the generated Playwright script using Python
- Uses subprocess.run for execution
- Runs synchronously and exits on failure

8. Execution Flow

Input Example

Write a Playwright test in Python to test the login functionality of https://practicetestautomation.com/practice-test-login/

Output Behavior

- A Playwright script is generated
- The script is saved as generated_playwright_test.py
- The browser is launched
- The test runs automatically
- The browser is closed at the end

10. Use Cases

- Automated test generation
- Rapid Playwright prototyping
- AI-assisted QA workflows
- Learning agentic code generation with LangGraph
- Demonstrating LLM-to-code execution pipelines

11. Conclusion

langgraph_playwright_codegen.py demonstrates how LangGraph can orchestrate complex, multi-step AI workflows, including code generation, file handling, and execution.

This project highlights the power of combining:

- LangGraph for control flow
- Gemini for intelligent code generation
- Playwright for browser automation

It serves as a strong foundation for building AI-driven testing and automation systems.